

SecureChat 安全双向通信系统实验报告

2026 年 6 月 14 日

目录

1	开发目的	3
2	系统功能	3
2.1	通信功能	3
2.2	安全功能	4
2.3	客户端和部署功能	5
3	系统设计	5
3.1	系统组成结构	5
3.2	主要数据结构	6
3.3	信令与数据通路设计	7
3.4	关键算法设计	7
3.4.1	X25519 与贡献式 GKA	7
3.4.2	文本和附件加密	8
3.4.3	pairwise 私发密钥	9
3.4.4	PKI 身份签名	9
3.4.5	TLS、WSS 与 mTLS	10
4	系统实现	10
4.1	开发环境	10
4.2	系统运行截图	11
4.3	关键代码实现	12
4.3.1	Server 监听与 mTLS backend 支持	12
4.3.2	Host/Client mTLS 客户端证书配置	12
4.3.3	信令字段白名单	13
4.3.4	GKA v3 贡献集合封装	14

目录	2
4.3.5 应用层 encrypted relay	15
4.3.6 pairwise 私发与错投检查	15
4.3.7 Host 房间治理	17
4.3.8 PKI join_room 身份绑定	18
4.3.9 Host 拒绝身份验证失败成员	19
4.3.10 PKI 验证结果同步到成员列表	20
4.3.11 房间密码摘要保存	21
4.3.12 WinUI 调用 native core	22
4.3.13 WinUI 附件预览安全策略	22
4.4 阶段十安全测试设计	24
5 项目开发总结	25
5.1 完成情况	25
5.2 当前系统缺陷和改进方向	26
5.3 对 Host 不可信场景的讨论	27
6 心得体会	27
7 参考文献	28

1 开发目的

本项目的开发目的，是设计并实现一个安全的双向通信工具。系统需要支持局域网和公网环境中的文本、图片、语音和文件通信，并尽可能降低不可信服务器、网络监听者和主动中间人对通信内容的威胁。

在传统客户端到服务器聊天模型中，服务器通常能够接触消息明文。SecureChat 的设计目标不是让服务器成为可信聊天成员，而是让服务器只承担房间注册、成员状态维护和密文转发职责。真正的聊天内容在成员端加密和解密，服务器只看到有限元数据和密文。

本项目重点解决以下问题：

1. 实现 Server、Host、Client 三角色分离，使 Server 不参与聊天语义，不创建群密钥，也不成为房间成员。
2. 使用端到端加密（End-to-End Encryption, E2EE）保护文本和附件内容。
3. 使用群组密钥协商（Group Key Agreement, GKA）模型，为房间成员分发对称群密钥。
4. 使用公钥基础设施（Public Key Infrastructure, PKI）成员身份认证，绑定成员身份和临时 X25519 公钥，降低主动中间人替换公钥的风险。
5. 支持传输层安全（Transport Layer Security, TLS）和可选双向传输层安全（mutual TLS, mTLS）部署，保护信令入口和连接准入。
6. 明确系统的安全边界：内容机密性、元数据可见性、恶意成员限制、附件本地缓存风险等。

因此，本系统不是单纯的聊天界面实验，而是围绕“安全的网络双向通信软件”进行的协议、工程和安全测试综合设计。

2 系统功能

SecureChat 当前实现的主要功能如下。

2.1 通信功能

- Host 创建房间，Client 加入房间。
- 同一个 Server 可以承载多个不同 roomId，同一 Server 内 roomId 不允许重复。
- 支持群发文本消息。

- 支持私发文本消息，发送目标可以是成员名或成员 id。
- 支持图片、语音和普通文件附件。
- 支持私发图片、语音和文件附件。
- 成员列表显示 name / id，便于选择私发目标。
- Host 支持禁言、解除禁言、驱逐和当前房间证书指纹封禁。
- WinUI 成员列表用绿色或红色身份框呈现验证状态，点击 name / id 框可复制完整证书指纹，用于附件自动预览策略。

2.2 安全功能

- 房间密码不再通过命令行参数传递，支持隐藏输入、标准输入管道和短生命周期环境变量。
- Server 只保存房间密码摘要，不保存房间密码明文。
- 文本和附件均通过 WebSocket encrypted relay 转发，Server 不解密应用层内容。
- GKA v3 使用临时 X25519 公钥、成员签名随机贡献和 Host 发起的 epoch。
- 成员加入或离开时，Host 发起新的 GKA epoch，当前成员重新导出 room group key。
- PKI 成员身份认证可以绑定 Client 的 join_room 公钥、GKA contribution 和 Host 的 group_key/group-state envelope。
- 私发文本和附件在外层 room relay 内使用 pairwise key，避免 Host 或其他成员解开目标成员私发内容。
- Host/Client 会丢弃 relayTargetId 不属于自己的私发 envelope，并缓存 relay nonce/tag 防止重放展示。
- WSS (WebSocket Secure) 支持服务器证书和 TLS 加密。
- mTLS 通过 Nginx 反向代理实现客户端证书准入。
- 附件具有扩展名、文件头、大小、文件名净化和缓存上限检查。
- WinUI 默认不自动预览 Unknown 或未 Trusted 的远端成员附件，图片和音频需先显示为附件卡片。

2.3 客户端和部署功能

- 提供 C++ 命令行 Server、Host、Client。
- 提供 Windows WinUI 桌面客户端。
- 提供 ASP.NET Core Web UI。
- 提供 Windows 构建脚本 build.bat、build_win.bat、build_web.bat。
- 提供 Linux 构建脚本 build.sh、build_web.sh。
- 提供 start_server.sh、start_host.sh、start_client.sh 及对应 stop 脚本。
- 提供可选 systemd 服务模板和 Nginx mTLS 反向代理模板。

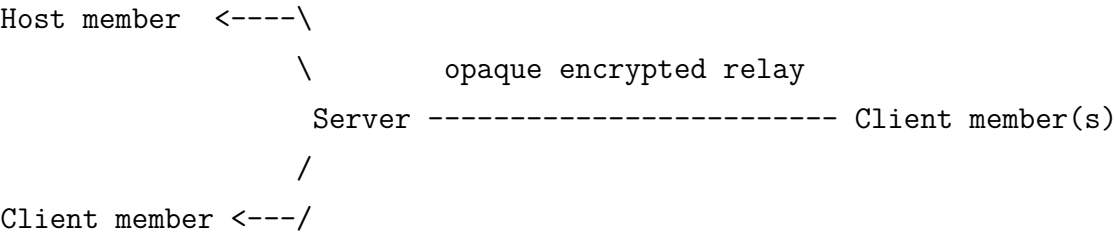
3 系统设计

3.1 系统组成结构

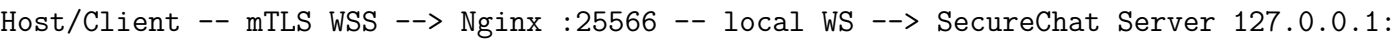
系统由三个协议角色和多个界面形态组成。

角色或组件	职责
Server	监听 WebSocket 端口，注册 room token，维护成员连接状态，校验房间访问 token，转发 encrypted relay、GKA contribution 和 group_key envelope。Server 不是群成员，不创建群密钥，不参与密钥协商语义。
Host	创建 roomId，是第一个群成员和房间生命周期管理者。Host 发起 GKA epoch，汇总签名贡献集合，处理成员加入、离开、驱逐时的密钥轮换，并执行禁言和当前房间证书指纹封禁。
Client	加入已有 room，生成临时 X25519 密钥对，参与 GKA v3，提交签名 contribution，使用 room group key 加解密群聊文本和附件，并使用已验证成员 public key 进行 pairwise 私发。
WinUI	Windows 桌面图形界面，通过 native.dll 调用 C++ 核心。
Web UI	ASP.NET Core Web 界面，通过 native.dll 或 libnative.so 调用 C++ 核心，并用 Server-Sent Events 显示消息。
Nginx mTLS proxy	可选部署组件，在 TLS 握手阶段验证客户端证书，再把 WebSocket 流量转发给本机 SecureChat Server backend。

系统拓扑如下：



在 mTLS 部署中，公网入口拓扑为：



3.2 主要数据结构

系统的运行期状态由以下内存结构维护。聊天消息和附件内容不保存在 Server 端，房间状态随进程和会话生命周期存在。

数据结构	说明
UserAccount	包含 userId 和 username，表示一个登录或注册后的成员身份。
AuthService::UserRecord	保存 UserAccount 和用户密码哈希，用于同一运行期内的用户名/密码校验。
RoomRegistry::RoomState	保存 room token、访问 token 摘要、Host 成员和 Client 成员映射。
SignalingServer::ClientState	保存单个 WebSocket 对应的 room token、clientId、userId、username、publicKey、identity、role 和坏消息计数。
SignalingServer::Room	保存 Host socket、Client socket 映射，以及 Host 请求的 silencedClients 当前房间发送限制集合。
HostSessionCore	Host 本地会话状态，包括 roomId、room token、username、Client 公钥表、当前 group key、GKA pending contribution、PKI identity context、当前房间封禁证书指纹集合和附件接收状态。
ClientSessionCore	Client 本地会话状态，包括 roomId、room token、临时 X25519 密钥对、当前 group key、PKI identity context、已验证成员 public key 映射、relay 重放缓存和附件接收状态。
IdentityContext	保存本机成员证书链、身份私钥、信任根和本地吊销列表。

3.3 信令与数据通路设计

信令消息使用 JSON 格式。主要类型包括：

- create_room: Host 创建房间。
- join_room: Client 加入房间, 提交临时 X25519 public key, 并可附带 PKI identity。
- new_client: Server 通知 Host 有新 Client 加入。
- gka_request: Host 发起新的 GKA epoch。
- gka_contribution: Client 向 Host 提交加密的签名随机贡献。
- group_key: Host 为目标 Client 单独封装完整 group state。
- encrypted_relay: Host/Client 之间的文本和附件密文 envelope。
- room_members: Server 广播当前成员 name/id, 并携带可被 Client 复验的 signed identity。
- reject_client: Host 拒绝身份验证失败的 Client。
- silence_client / unsilence_client: Host 请求 Server 暂停或恢复某个 Client 的 encrypted relay 发送权限。

Server 对每种信令类型进行字段白名单检查和大小限制。Server 会用 WebSocket 会话状态覆盖 sender metadata, 避免普通成员伪造 senderId 或跨房间注入。

3.4 关键算法设计

3.4.1 X25519 与贡献式 GKA

X25519 是基于 Curve25519 的 Diffie-Hellman (DH) 函数。设椭圆曲线基点为 G , Client 生成临时私钥 c , 公钥为:

$$C = cG$$

Host 为该 Client 生成一次性 group-state 封装私钥 e , 公钥为:

$$E = eG$$

双方计算共享秘密:

$$S_H = eC = ecG$$

$$S_C = cE = ceG$$

由于椭圆曲线 DH 性质，二者相等：

$$S_H = S_C$$

共享秘密不直接作为加密密钥，而是经过基于 HMAC 的密钥派生函数（HMAC-based Key Derivation Function, HKDF）派生 group-state 包装密钥：

$$K_W = \text{HKDF-SHA256}(S, \text{salt} = \text{"securechat-gka-state-v3:"} || \text{roomToken}, \text{info} = \text{"group-state"} || \text{client})$$

每个成员 i 为当前 epoch 生成 32 字节随机贡献 r_i ，并使用成员身份私钥签名：

$$\sigma_i = \text{Sign}(sk_i, \text{roomId} || \text{epoch} || \text{memberId}_i || \text{username}_i || X_i || r_i || \text{nonce}_i)$$

Host 汇总贡献集合 $C = \{(\text{memberId}_i, X_i, r_i, \sigma_i)\}_{i=1}^n$ ，把完整 group state 用 K_W 封装给每个 Client：

$$(C_s, T_s) = \text{AES-256-GCM.Enc}(K_W, N, \text{groupState}, A)$$

其中 N 是随机 nonce， A 是附加认证数据（Additional Authenticated Data, AAD）。AAD 绑定 room token、targetId 和 epoch，防止 Server 静默修改 group_key 的目标成员或 epoch。Client 解开 group state 后验证每个 σ_i ，再本地导出 room group key：

$$K_G = \text{HKDF-SHA256}(\text{Canonical}(C), \text{salt} = \text{"securechat-gka-v3:"} || \text{roomToken} || \text{epoch}, \text{info} = \text{"room-g"})$$

3.4.2 文本和附件加密

Host/Client 得到同一个 K_G 后，文本、图片 metadata、文件 metadata、语音 metadata 以及二进制 chunk 都使用 AES-256-GCM 加密：

$$(C, T) = \text{AES-256-GCM.Enc}(K_G, N, P, A)$$

其中 P 是明文 JSON 或附件二进制块， C 是密文， T 是认证标签。AAD 绑定 room token 和 senderId；应用层 senderName、senderKind 和 targetId 均位于加密 payload 内：

$$A = \text{"securechat-relay-v3"} || \text{roomToken} || \text{senderId}$$

这意味着攻击者如果修改 room token 或 senderId, 接收端重新构造 AAD 后会导致 GCM 认证失败。私发 targetId 由接收端在解密 payload 后检查, 目标不是自己时丢弃。

3.4.3 pairwise 私发密钥

私发不是只依赖 K_G 。发送者 A 为每条私发生成一次性 X25519 私钥 e_A , 目标成员 B 的已验证 X25519 public key 记为 X_B 。双方共享秘密为:

$$S_{AB} = \text{X25519}(e_A, X_B)$$

系统再通过 HKDF 绑定房间、发送者、目标成员和双方证书指纹:

$$K_{AB} = \text{HKDF-SHA256}(S_{AB}, \text{salt} = \text{"securechat-pairwise-v1:"} || \text{roomId} || A || B, \text{info} = \text{"private-message"})$$

其中 E_A 是发送者一次性 public key, fp_A 和 fp_B 是发送者与目标成员证书 SHA-256 指纹。私发明文 P 再用 K_{AB} 加密:

$$(C, T) = \text{AES-256-GCM.Enc}(K_{AB}, N, P, A_{\text{pairwise}})$$

pairwise 密钥不从 K_G 派生。因此 Server、Host 或其他成员即使拿到外层 room relay, 也不能解开内层私发正文或附件 chunk。

3.4.4 PKI 身份签名

PKI 模式用于解决“这个临时 X25519 public key 到底属于谁”的问题。Client 在 join_room 中签名:

$$M_J = \text{roomId} || \text{username} || \text{publicKey} || \text{nonce}$$

$$\sigma_J = \text{Sign}(sk_C, M_J)$$

Host 使用成员证书中的签名公钥验证:

$$\text{Verify}(pk_C, M_J, \sigma_J) = 1$$

成员提交 GKA contribution 时签名:

$$M_C = \text{roomId} || \text{epoch} || \text{memberId} || \text{username} || \text{publicKey} || \text{contribution} || \text{nonce}$$

$$\sigma_C = \text{Sign}(sk_C, M_C)$$

Host 发送 group_key/group-state envelope 时签名:

$$M_G = version || roomId || epoch || targetId || senderId || alg || kdf || ephemeralPublicKey || nonce || ciphertext$$

$$\sigma_G = \text{Sign}(sk_H, M_G)$$

Client 先验证 Host 证书链、吊销状态和签名, 再解封装 group state。解开后还要验证每个成员的 σ_C , 再导出 group key。这样恶意 Server 不能静默替换 Client public key、伪造成员 contribution, 也不能静默篡改 group_key envelope。

3.4.5 TLS、WSS 与 mTLS

WebSocket (WS) 是明文 WebSocket; WebSocket Secure (WSS) 是在 TLS 上承载 WebSocket。WSS 保护 Host/Client 到 Server 的传输层, 防止被动监听者直接看到信令 JSON。

mTLS 是 mutual TLS, 即 TLS 双向认证。普通 TLS 只要求 Client 验证 Server 证书; mTLS 同时要求 Server 入口验证 Client 证书。由于 libdatachannel 当前没有暴露 WebSocketServer 客户端证书校验接口, 本系统使用 Nginx 在反向代理层实现 mTLS:

$$\text{Client} \xrightarrow{\text{mTLS}} \text{Nginx} \xrightarrow{\text{local WS}} \text{SecureChat Server}$$

mTLS 用于连接准入, PKI 成员身份签名用于 GKA 语义绑定, 二者不是同一个安全检查点。Host/Client 通过 SECURECHAT_MTLS_CLIENT_CERT_FILE 和 SECURECHAT_MTLS_CLIENT_KEY_FILE 配置客户端证书, 在 TLS 握手时出示给 Nginx。若服务器证书由私有 CA 或自签名证书签发, 则额外使用 SECURECHAT_TLS_CA_FILE 指定信任根。

4 系统实现

4.1 开发环境

类别	环境
开发系统	Windows x64, PowerShell, Visual Studio 2026 Developer Command Prompt。
云部署系统	Linux x64, 面向公网 Server 和 Web UI 部署。

C++ 标准	C++17。
构建系统	CMake、Ninja、vcpkg。
主要 C++ 库	libdatachannel、OpenSSL、nlohmann-json。
桌面 UI	Windows WinUI, .NET 10。
Web UI	ASP.NET Core, .NET 10, Server-Sent Events。
抓包工具	Wireshark、tcpdump。
部署工具	systemd、Nginx、Certbot 或其他 ACME 客户端。

Windows 构建命令：

```
1 build_win.bat
2 build_web.bat
```

Linux 构建命令：

```
1 ./build.sh
2 ./build_web.sh
```

4.2 系统运行截图

本节截图在提交最终报告前补充。



图 1: WinUI Host 创建房间



图 2: WinUI Client 加入房间

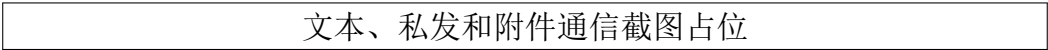


图 3: 文本、私发和附件通信



图 4: Wireshark 显式安全性验证

4.3 关键代码实现

4.3.1 Server 监听与 mTLS backend 支持

以下代码展示 Server 的监听配置。普通模式监听所有网卡；mTLS 反向代理模式下设置 SECURECHAT_BIND_ADDRESS=127.0.0.1，使 SecureChat Server 只作为本机 backend 被 Nginx 访问。

Listing 1: signaling_server.cpp: Server 监听配置

```

1 SignalingServer::SignalingServer(uint16_t port) {
2     rtc::WebSocketServer::Configuration config;
3     config.port = port;
4
5     // mTLS 部署时只监听 127.0.0.1，公网入口交给 Nginx。
6     const auto bindAddress = envValue("SECURECHAT_BIND_ADDRESS");
7     config.bindAddress = bindAddress.empty() ? "0.0.0.0" : bindAddress;
8
9     // 防止半开 WebSocket 握手长期占用公网入口。
10    config.connectionTimeout = std::chrono::seconds(15);
11
12    if (envEnabled("SECURECHAT_SIGNALING_TLS")) {
13        const auto certFile = envValue("SECURECHAT_TLS_CERT_FILE");
14        const auto keyFile = envValue("SECURECHAT_TLS_KEY_FILE");
15        if (certFile.empty() || keyFile.empty()) {
16            throw std::runtime_error("TLS cert and key are required");
17        }
18        config.enableTls = true;
19        config.certificatePemFile = certFile;
20        config.keyPemFile = keyFile;
21        mUrlScheme = "wss";
22    }
23 }
```

4.3.2 Host/Client mTLS 客户端证书配置

Host/Client 使用同一段 WebSocket 客户端配置逻辑。配置 SECURECHAT_TLS_CA_FILE 可以指定自定义 CA，配置 SECURECHAT_MTLS_CLIENT_CERT_FILE 和 SECURECHAT_MTLS_CLIENT_KEY_FILE 后，客户端会在 TLS 握手中出示证书。

Listing 2: websocket_config.cpp: mTLS 客户端证书配置

```

1 void applyClientTlsFromEnvironment(rtc::WebSocket::Configuration& config) {
```

```

2  const auto caFile = envValue("SECURECHAT_TLS_CA_FILE");
3  const auto certFile = envValue("SECURECHAT_MTLS_CLIENT_CERT_FILE");
4  const auto keyFile = envValue("SECURECHAT_MTLS_CLIENT_KEY_FILE");
5  const auto keyPass = envValue("SECURECHAT_MTLS_CLIENT_KEY_PASS");
6
7  if (!caFile.empty()) {
8      config.caCertificatePemFile = caFile;
9  }
10
11  if (!certFile.empty() || !keyFile.empty() || !keyPass.empty()) {
12      if (certFile.empty() || keyFile.empty()) {
13          throw std::runtime_error("mTLS client cert and key are required");
14      }
15      config.certificatePemFile = certFile;
16      config.keyPemFile = keyFile;
17      if (!keyPass.empty()) {
18          config.keyPemPass = keyPass;
19      }
20  }
21 }

```

4.3.3 信令字段白名单

Server 对每种信令消息做字段白名单和大小限制。这样未知字段不能悄悄变成协议扩展或伪造通道。

Listing 3: signaling_server.cpp: identity 字段校验

```

1  void identityField(const json& data, bool required) {
2      auto it = data.find("identity");
3      if (it == data.end()) {
4          if (required) throw std::runtime_error("missing identity");
5          return;
6      }
7      if (!it->is_object()) throw std::runtime_error("identity must be an object");
8      const auto& identity = *it;
9
10     // Server 只做结构和大小检查, 证书语义由 Host/Client 本地验证。
11     if (!hasOnlyFields(identity,
12         {"version", "certChainPem", "nonce", "signatureAlg", "signature"})) {

```

```

13     throw std::runtime_error("identity has unknown field");
14 }
15 stringField(identity, "certChainPem", maxIdentityCertChainBytes, true);
16 stringField(identity, "nonce", 64, true);
17 stringField(identity, "signatureAlg", 32, true);
18 stringField(identity, "signature", maxIdentitySignatureBytes, true);
19 }

```

4.3.4 GKA v3 贡献集合封装

Host 为每个 Client 单独生成一次性 X25519 密钥对，并派生 wrapping key 封装完整 group state。Client 解开后验证 contribution set 并本地导出 K_G 。

Listing 4: secure_relay.cpp: 单成员 group state 封装

```

1 json encryptGroupStateForMember(
2     const json& groupState,
3     const std::string& roomId,
4     const std::string& targetId,
5     const std::string& targetPublicKey,
6     std::uint64_t epoch) {
7     // Server 只转发 envelope, 不能由 targetPublicKey 推导 wrapping key.
8     const auto recipientPublic = base64Decode(targetPublicKey);
9     auto ephemeral = generateMemberKeyPair();
10    const auto secret = deriveX25519Secret(ephemeral.privateKey,
11        recipientPublic);
12    const auto wrapKey = hkdfSha256(
13        secret,
14        "securechat-gka-state-v3:" + roomId,
15        "group-state|" + targetId + "|" + std::to_string(epoch));
16    return encryptWithAesGcm(groupState.dump(), wrapKey,
17        aadForGroupKey(roomId, targetId, epoch), {
18        {"type", GroupKeyType},
19        {"version", 3},
20        {"epoch", epoch},
21        {"roomId", roomId},
22        {"targetId", targetId},
23        {"senderId", chat::protocol::HostActorId},
24        {"alg", "AES-256-GCM"},
25        {"kdf", "X25519-HKDF-SHA256+GKA-Contrib-v3"},

```

```

26     {"ephemeralPublicKey", ephemeral.publicKey}
27     });
28 }

```

4.3.5 应用层 encrypted relay

文本和附件都封装为 encrypted_relay。应用层 senderName、senderKind 和 targetId 不再作为明文字段出现；私发 targetId 位于加密 payload 内，Server 只广播外层密文。

Listing 5: secure_relay.cpp: 文本和附件 relay 加密

```

1 json encryptMessageWithGroupKey(
2     const Message& message,
3     const std::string& roomId,
4     const std::string& senderId,
5     const std::string& senderName,
6     const std::string& senderKind,
7     const std::string& targetId,
8     const std::vector<unsigned char>& groupKey) {
9     // senderName/senderKind/targetId 不明文发送，Server 只看到 room token 和
10    senderId。
11    const auto key = keyArrayFromVector(groupKey);
12    const auto aad = aadForEnvelope(roomId, senderId, senderKind, targetId);
13
14    return encryptWithAesGcm(message.toJson(), key, aad, {
15        {"type", EnvelopeType},
16        {"version", 3},
17        {"roomId", roomId},
18        {"senderId", senderId},
19        {"alg", "AES-256-GCM"},
20        {"kdf", "GKA-Contrib-v3-HKDF-SHA256"}
21    });
22 }

```

4.3.6 pairwise 私发与错投检查

私发先被封装为 ‘pairwise_private’ 内层消息，再作为普通 encrypted_relay 发送。发送端必须先取得目标成员已验证 public key，否则直接失败，不会降级为只使用 room group key 的私发。

Listing 6: secure_relay.cpp: 私发内层 pairwise 加密

```

1 Message encryptMessageForPairwise(
2     const Message& message,
3     const std::string& roomId,
4     const std::string& senderId,
5     const std::string& targetId,
6     const std::string& targetPublicKey,
7     const std::string& senderFingerprint,
8     const std::string& targetFingerprint) {
9     // 不从 K_G 派生 pairwise key; 每个群成员都有 K_G。
10    auto ephemeral = generateMemberKeyPair();
11    const auto secret = deriveX25519Secret(
12        ephemeral.privateKey, base64Decode(targetPublicKey));
13    const auto pairwiseKey = hkdfSha256(
14        secret,
15        "securechat-pairwise-v1:" + roomId + "|" + senderId + "|" + targetId,
16        "private-message|" + senderFingerprint + "|" +
17            targetFingerprint + "|" + ephemeral.publicKey);
18    const auto inner = encryptWithAesGcm(
19        message.toJson(),
20        pairwiseKey,
21        aadForPairwise(roomId, senderId, targetId,
22            senderFingerprint, targetFingerprint, ephemeral.publicKey),
23        {{ "type", "pairwise_private" }});
24    Message wrapper;
25    wrapper.type = PairwisePrivateType;
26    wrapper.payload["pairwise"] = inner;
27    return wrapper;
28 }

```

为了避免恶意 Server 把私发 envelope 错投给诚实成员后被 UI 展示, Host 和 Client 先检查外层 relayTargetId; 目标不是自己时直接丢弃。即使攻击者修改客户端跳过该检查, 内层 pairwise GCM 认证也需要目标成员私钥。

Listing 7: client_session_core.cpp: 丢弃错误投递并打开 pairwise 私发

```

1 if (!rememberRelayEnvelope(j)) return;
2 auto msg = chat::secure_relay::decryptMessageWithGroupKey(
3     j, mRoomToken, mGroupKey);
4 const auto relayTargetId = msg.payload.value("relayTargetId", "");
5 if (!relayTargetId.empty() && relayTargetId != mClientId) {
6     chatEmit(mCallbacks.onError,

```



```

7     "Dropped encrypted relay for another member");
8     return;
9 }
10 if (msg.type == chat::secure_relay::PairwisePrivateType) {
11     msg = decryptPairwiseFromMember(msg);
12 }
13 handleRelayMessage(msg);

```

4.3.7 Host 房间治理

Host 管理命令不会进入聊天历史，而是在本地解析为管理操作。silence 只改变发送权限，不改变成员资格；evict/ban 会关闭目标 Client，并在当前房间生命周期内记录其证书指纹。

Listing 8: host_session_core.cpp: Host 管理命令入口

```

1 bool HostSessionCore::handleHostCommand(const std::string& line) {
2     std::istringstream input(trimCopy(line));
3     std::string command;
4     input >> command;
5     if (command != "/silence" &&
6         command != "/unsilence" &&
7         command != "/evict" &&
8         command != "/ban") {
9         return false;
10    }
11
12    std::string target;
13    input >> target;
14    if (target.empty()) {
15        chatEmit(mCallbacks.onError,
16                "Usage: " + command + " <member-name-or-id>");
17        return true;
18    }
19
20    if (command == "/silence") setClientSilenced(target, true);
21    else if (command == "/unsilence") setClientSilenced(target, false);
22    else evictClient(target);
23    return true;
24 }

```

Listing 9: signaling_server.cpp: Server 执行禁言发送限制

```

1 if (sender->role == "client" &&
2     room->silencedClients.find(senderId) !=
3     room->silencedClients.end()) {
4     errorMessage = "member is silenced";
5 }
6 else {
7     envelope["roomId"] = roomId;
8     envelope["senderId"] = senderId;
9     envelope["senderName"] = sender->username;
10    envelope["senderKind"] = senderKind;
11    envelope["targetId"] = targetId;
12    // 后续再按 targetId 选择 room broadcast 或 private recipient。
13 }

```

4.3.8 PKI join_room 身份绑定

Client 对 roomId、username、临时 X25519 publicKey 和 nonce 签名。Host 验证证书链、吊销状态和签名后才接受该 publicKey。

Listing 10: identity_pki.cpp: join_room 签名与验证

```

1 json IdentityContext::signJoinRoom(
2     const std::string& roomId,
3     const std::string& username,
4     const std::string& publicKey) const {
5     if (!mData) throw std::runtime_error("PKI identity is not configured");
6     const auto nonce = randomNonce();
7     return makeIdentityObject(
8         mData->privateKey.get(),
9         mData->certChainPem,
10        canonicalJoinMessage(roomId, username, publicKey, nonce),
11        nonce);
12 }
13
14 VerifiedIdentity IdentityContext::verifyJoinRoom(
15     const std::string& roomId,
16     const std::string& username,
17     const std::string& publicKey,
18     const json& identity) const {
19     const auto nonce = requiredIdentityString(identity, "nonce");

```

```

20     const auto signature = base64Decode(requiredIdentityString(identity, "
21         signature"));
22     const auto certs = verifiedIdentityCerts(
23         mData->trustStore.get(),
24         mData->revokedFingerprints,
25         identity);
26     EvpPkeyPtr publicSigningKey(X509_get_pubkey(certs.front().get()),
27         EVP_PKEY_free);
28     requireSignatureAlgorithmMatches(publicSigningKey.get(), identity);
29     if (!verifyBytes(publicSigningKey.get(),
30         canonicalJoinMessage(roomId, username, publicKey, nonce),
31         signature)) {
32         throw std::runtime_error("join_room identity signature verification
33             failed");
34     }
35     return {
36         certificateSubject(certs.front().get()),
37         certificateFingerprint(certs.front().get())
38     };

```

4.3.9 Host 拒绝身份验证失败成员

Host 发现成员身份验证失败时，不记录该成员 publicKey，也不轮换 group key，而是请求 Server 移除该 Client。

Listing 11: host_session_core.cpp: Host 验证并拒绝 Client

```

1  if (!clientId.empty()) {
2      try {
3          auto identity = j.find("identity");
4          if (identity == j.end()) throw std::runtime_error("client identity is
5              missing");
6          const auto verified = mIdentity.verifyJoinRoom(
7              mRoomId, username, publicKey, *identity);
8          identityFingerprint = verified.fingerprint;
9          identitySubject = verified.subject;
10         chatEmit(mCallbacks.onStatus,
11             "PKI member verified: " + username + " / " +

```

```

11         clientId + " / " + identityFingerprint);
12     }
13     catch (const std::exception& e) {
14         chatEmit(mCallbacks.onError,
15             "Client identity rejected: " + username + ": " + e.what());
16         rejectClient(clientId, "client identity verification failed");
17         return;
18     }
19 }

```

4.3.10 PKI 验证结果同步到成员列表

Client 入房时提交的 signed identity 会出现在 Server 的 room_members.memberInfos 中，其他 Client 复验后才把 member id/name 映射到 pairwise public key。Host 验证 Client 证书和签名后，也会把成员证书指纹、publicKey 和原始 signed identity 作为加密控制消息同步给房间成员。Server 只转发 encrypted relay，不能读取该控制消息内容；接收端仍会重新验证 identity，并拒绝同一 member id 上的公钥或指纹冲突。

Listing 12: host_session_core.cpp: 广播已验证成员 identity

```

1 void HostSessionCore::announceVerifiedMember(
2     const std::string& memberId,
3     const std::string& displayName,
4     const std::string& fingerprint,
5     const std::string& subject,
6     const std::string& publicKey,
7     const json& identity) {
8     if (memberId.empty() || fingerprint.empty() ||
9         publicKey.empty() || !identity.is_object()) return;
10
11     Message msg;
12     msg.type = "member_identity";
13     msg.from = currentHostActorName();
14     setCurrentHostActorMetadata(msg);
15     msg.payload["memberId"] = memberId;
16     msg.payload["displayName"] = displayName.empty() ? memberId : displayName;
17     msg.payload["fingerprint"] = fingerprint;
18     msg.payload["subject"] = subject;
19     msg.payload["publicKey"] = publicKey;
20     msg.payload["identity"] = identity;

```

```

21 msg.payload["state"] = "verified";
22 msg.payload["verifiedBy"] = chat::protocol::HostActorId;
23
24 sendRelayMessage(
25     msg,
26     chat::protocol::HostActorId,
27     mUsername,
28     chat::protocol::HostActorKind,
29     "");
30 }

```

4.3.11 房间密码摘要保存

RoomRegistry 不保存房间密码明文，只保存带 domain separation 的 SHA-256 摘要。比较时使用常量时间比较。

Listing 13: room_registry.cpp: 房间密码摘要

```

1 std::array<unsigned char, 32> hashRoomPassword(
2     const std::string& roomId,
3     const std::string& password) {
4     // 只保存摘要，避免房间密码作为长生命周期明文留在 Server 内存中。
5     std::array<unsigned char, 32> digest{};
6     EVP_DigestInit_ex(ctx.get(), EVP_sha256(), nullptr);
7     EVP_DigestUpdate(ctx.get(), "securechat-room-v1", 18);
8     EVP_DigestUpdate(ctx.get(), roomId.data(), roomId.size());
9     EVP_DigestUpdate(ctx.get(), password.data(), password.size());
10    EVP_DigestFinal_ex(ctx.get(), digest.data(), &digestLength);
11    return digest;
12 }
13
14 bool RoomRegistry::passwordMatches(
15     const std::string& roomId,
16     const std::string& password) const {
17     const auto suppliedHash = hashRoomPassword(roomId, password);
18     return CRYPTO_memcmp(
19         storedHash.data(),
20         suppliedHash.data(),
21         storedHash.size()) == 0;
22 }

```

4.3.12 WinUI 调用 native core

WinUI 不直接实现密码协议，而是调用 native.dll 中的 C++ 核心。这样可以使 CLI、WinUI 和 Web UI 共用一套安全实现。

Listing 14: MainWindow.xaml.cs: Host 和 Join 入口

```

1 private void Host_Click(object sender, RoutedEventArgs e)
2 {
3     // Host 是可见群成员；独立 Server 负责监听和 TLS。
4     var ok = NativeMethods.chat_host_start(
5         HostServerUrlBox.Text.Trim(),
6         HostRoomBox.Text.Trim(),
7         HostUserBox.Text.Trim(),
8         HostPasswordBox.Password);
9     if (ok != 0) {
10         SetSessionMode(SessionMode.Host);
11     }
12 }
13
14 private void Join_Click(object sender, RoutedEventArgs e)
15 {
16     var ok = NativeMethods.chat_join_start(
17         JoinUrlBox.Text.Trim(),
18         JoinRoomBox.Text.Trim(),
19         JoinUserBox.Text.Trim(),
20         JoinPasswordBox.Password);
21     if (ok != 0) {
22         SetSessionMode(SessionMode.Join);
23     }
24 }

```

4.3.13 WinUI 附件预览安全策略

WinUI 使用 Unknown、Verified、Trusted、Blocked 四态控制附件预览。Unknown 表示没有 PKI 验证结果；Verified 表示成员证书和签名已通过 PKI 验证；Trusted 表示用户在当前房间内允许该 Verified 成员自动预览附件；Blocked 表示本机用户阻止该成员自动预览。成员列表只显示 name / id，Verified/Trusted 使用绿色身份框，Unknown/Blocked 使用红色身份框；点击身份框会复制完整证书指纹。Trusted 和 Blocked 都是当前房间内存状态，不跨房间保存。

Listing 15: MainWindow.xaml.cs: 附件预览策略

```

1 // These caps only protect local WinUI preview/decoder paths.
2 // The protocol transfer limit remains SECURECHAT_ATTACHMENT_MAX_BYTES.
3 private const long MaxPreviewImageBytes = 15L * 1024 * 1024;
4 private const long MaxPreviewAudioBytes = 50L * 1024 * 1024;
5 private const int MaxPreviewImageDimension = 8192;
6 private const long MaxPreviewImagePixels = 24_000_000;
7 private const double MaxPreviewAudioSeconds = 600;
8
9 private bool ShouldAutoPreviewAttachment(AttachmentPreviewInfo info)
10 {
11     if (info.Kind == "file") return false;
12     if (info.IsOwnLocalAttachment) {
13         return info.Kind == "image" ? autoPreviewImages : autoLoadAudio;
14     }
15     if (info.MemberState is AttachmentMemberState.Unknown or
16         AttachmentMemberState.Blocked) return false;
17     if (onlyTrustedAttachmentPreview &&
18         info.MemberState != AttachmentMemberState.Trusted) return false;
19     return info.Kind == "image" ? autoPreviewImages : autoLoadAudio;
20 }
21
22 private void ToggleTrustedParticipant(string participant)
23 {
24     var key = PrimaryParticipantKey(participant);
25     var state = MemberStateForParticipant(participant);
26     if (state == AttachmentMemberState.Trusted) {
27         trustedAttachmentMembers.Remove(key);
28     } else if (state == AttachmentMemberState.Verified) {
29         blockedAttachmentMembers.Remove(key);
30         trustedAttachmentMembers.Add(key);
31     }
32     RefreshParticipants();
33 }
34
35 private void ValidateImagePreview(string path, long sizeBytes)
36 {
37     if (sizeBytes > MaxPreviewImageBytes) {
38         throw new InvalidDataException("image is too large for inline preview")

```

```

        ;
39     }
40     var image = ReadImagePreviewInfo(path);
41     if (image.Width > MaxPreviewImageDimension ||
42         image.Height > MaxPreviewImageDimension ||
43         (long)image.Width * image.Height > MaxPreviewImagePixels) {
44         throw new InvalidDataException("image exceeds preview limit");
45     }
46 }
47
48 private void ValidateWavPreview(string path, long sizeBytes)
49 {
50     if (sizeBytes > MaxPreviewAudioBytes) {
51         throw new InvalidDataException("audio is too large for inline preview");
52     };
53
54     var wav = ReadWavPreviewInfo(path);
55     if (wav.Channels is not (1 or 2) ||
56         wav.SampleRate < 8000 || wav.SampleRate > 192000 ||
57         wav.DurationSeconds > MaxPreviewAudioSeconds) {
58         throw new InvalidDataException("unsupported WAV preview parameters");
59     }
60 }
```

4.4 阶段十安全测试设计

阶段十的测试分为本机可完成测试和需要服务器/抓包工具手动完成的测试。

测试项	预期现象	状态
build_win.bat	C++、native.dll、WinUI 构建通过，无错误。	已完成。
build_web.bat	Web UI 和 native core 构建通过，无错误。	已完成。
旧连接层扫描	不存在 Host/Client DataChannel 建连逻辑；STUN/ICE 只在已移除语境出现。	已完成。

WS 抓包	可见 join_room 和房间密码，但不可见聊天明文和附件内容。	手动执行。
WSS 抓包	只能看到 TLS record 和元数据，不能看到 WebSocket JSON。	手动执行。
mTLS 准入	无客户端证书连接失败，有受信任证书连接成功。	手动执行。
PKI publicKey 替换	篡改 publicKey 或 identity 后 Host/Client 拒绝；成员列表公钥冲突不覆盖已验证映射。	手动执行。
房间治理、错误投递和重放	禁言阻断发送，驱逐触发重密钥；错投私发不展示且 pairwise 内层不可解；旧 relay/group_key 被拒绝，未知 client_left 被忽略。	手动执行。
附件安全	伪装文件、超大文件和路径穿越文件名被拒绝或净化。	手动执行。
端口扫描和 TCP 半连接	公网只暴露必要端口；半连接防护由系统、云安全组和反向代理承担。	手动执行。

详细步骤已经整理在 security-tests.md 中。需要服务器和 Wireshark 的实验保留为手动步骤，是因为它们依赖真实网卡、云安全组、Nginx 证书和抓包截图。

5 项目开发总结

5.1 完成情况

本项目已经完成了从“Host 同时承担信令服务”到“Server/Host/Client 三角色分离”的重构。当前 Server 是不可信信令协调者和密文 relay，不是聊天成员。Host 是第一个群成员和房间生命周期管理者，Client 是普通成员，群聊使用贡献式 GKA v3 导出的 room group key 加解密文本和附件，私发使用额外 pairwise key 保护目标成员内容。

当前主要安全成果包括：

- 移除 WebRTC/DataChannel 数据通路，统一为 TCP WebSocket encrypted relay。

- 实现 GKA v3: Client 临时 X25519 public key, 成员签名随机 contribution, Host 发起 epoch 并封装 group state。
- 实现成员加入/离开后的 GKA epoch 更新和 group key rotation。
- 实现文本、图片、语音、文件 metadata 和 binary chunk 的 AES-256-GCM 加密。
- 实现 pairwise 私发密钥, 私发文本和附件不再只依赖 room group key。
- 实现 PKI 成员身份认证, 绑定 join_room publicKey、GKA contribution 和 group_key/group-state envelope。
- 实现成员列表 signed identity 复验和成员公钥冲突拒绝。
- 实现本地证书指纹吊销列表。
- 实现 Host 禁言、解除禁言、驱逐和当前房间证书指纹封禁。
- 实现错误投递私发的接收端 relayTargetId 检查、relay 重放缓存、group_key epoch 检查、未知 client_left 的忽略处理, 以及普通 Server error 不触发 Client 主动退出。
- 实现 WSS 和 mTLS 反向代理部署路径。
- 完成 WinUI 和 Web UI 的基本可用界面。

5.2 当前系统缺陷和改进方向

当前系统仍有一些重要限制。

第一, Host 在当前 GKA v3 中不再单方生成 room group key, 但仍是房间生命周期管理者和 epoch 发起者。如果 Host 不可信, Host 仍可以拒绝成员加入、驱逐成员、关闭房间或影响可用性。恶意成员也可以拒绝提交 GKA contribution, 从而延迟新 epoch; 当前实现加入了 10 秒 GKA contribution watchdog, 超时后 Host 自动驱逐未贡献成员, 并用剩余成员重新发起 epoch。pairwise 私发可以防止 Host 仅凭群聊 K_G 解开目标成员私发内容, 但不能阻止 Host 作为管理者影响房间状态。

第二, Server 仍能观察元数据, 包括 room token、连接 id、成员在线状态、消息大小、发送频率和转发时序。E2EE 保护内容机密性, 但不隐藏通信模式。后续可以考虑 padding、批量发送或混淆转发时序, 但会增加带宽和延迟成本。

第三, 附件安全仍不等于恶意文件防护。当前系统能检查大小、扩展名、文件头、文件名和缓存目录, WinUI 也默认不自动预览未知成员附件, 并在预览前检查图片尺寸、像素数和 WAV 结构; 但这些措施不能替代杀毒、沙箱和复杂文档解析隔离。

第四, 当前 PKI 吊销使用本地受信任指纹列表。它简单可复现, 但不是完整在线证书状态协议 (Online Certificate Status Protocol, OCSP) 或证书吊销列表 (Certificate

Revocation List, CRL) 体系。正式部署可进一步接入自动化证书签发、吊销分发和密钥轮换流程。

第五, Server 仍可造成可用性和成员状态层影响。它不能静默伪造 PKI 签名, 也不能让诚实端展示错误投递的私发; 普通 relay error 也不会让 Client 主动退出。但它仍可以断开连接、丢弃消息, 或对真实已知成员制造离线事件。当前系统会对未知 client_left 保守忽略, 但无法把不可信 Server 的断连行为变成可信事实。

5.3 对 Host 不可信场景的讨论

如果 Host 不可信, 当前系统已经避免 Host 单方生成 K_G , 因为 K_G 由成员签名 contribution set 导出。PKI 可以证明“某个 public key 和 contribution 属于某个成员身份”, mTLS 可以限制“谁能连接入口”。但是 Host 仍是房间创建者、关闭者和 GKA epoch 发起者, 可以通过管理动作影响房间可用性。对于私发, 当前 pairwise 内层密钥可以让 Host 不能仅凭 K_G 解开 Client 与 Client 之间的私发正文或附件。

若要对不可信 Host 建立更强安全性, 可以考虑:

- MLS 协议, 使用 TreeKEM 等机制管理群成员变更。
- 增加透明日志或成员可验证的密钥变更记录, 降低 Host 静默作恶空间。

6 心得体会

通过本项目可以看到, 安全通信软件的难点不仅是“把消息加密”, 更是把系统边界讲清楚并落到代码中。最初如果 Host 同时承担信令服务和聊天成员职责, 系统很容易在概念上混淆: 到底谁可信、谁能看明文、谁负责密钥协商, 都不够清晰。将 Server、Host、Client 拆开后, 安全模型才变得可解释。

另一个体会是, TLS、mTLS、PKI 和 E2EE 分别解决不同问题。TLS 保护传输通道, mTLS 做连接准入, PKI 身份签名绑定成员身份和临时公钥, E2EE 保护应用内容。它们可以叠加, 但不能互相替代。设计安全系统时, 如果把这些概念混在一起, 就很容易产生“用了 TLS 就等于端到端加密”之类的错误判断。

本项目也体现了工程取舍的重要性。例如 libdatachannel 已经提供 WebSocket 和 TLS 能力, 但没有直接暴露服务端客户端证书验证接口。强行修改底层库成本较高, 使用 Nginx 做 mTLS 反向代理则更稳定、更容易部署和测试。这说明安全工程中并不总是要把所有能力都塞进应用进程, 清晰的分层往往更可靠。

最后, 安全测试同样重要。Wireshark 抓包可以直观看到 WS 明文信令与 WSS/mTLS 加密传输的区别; public key 替换挑战可以说明为什么 X25519 本身不提供身份认证; 附件伪装和路径穿越测试可以提醒我们文件安全不是单一加密算法能解决的问题。这些实验能帮助系统从“功能可用”走向“安全边界可证明”。

7 参考文献

参考文献

- [1] I. Fette and A. Melnikov. The WebSocket Protocol. RFC 6455, 2011.
- [2] E. Rescorla. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446, 2018.
- [3] A. Langley, M. Hamburg, and S. Turner. Elliptic Curves for Security. RFC 7748, 2016.
- [4] H. Krawczyk and P. Eronen. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). RFC 5869, 2010.
- [5] National Institute of Standards and Technology. Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. NIST SP 800-38D, 2007.
- [6] OpenSSL Project. OpenSSL Documentation. <https://www.openssl.org/docs/>
- [7] Nginx. Module ngx_http_ssl_module Documentation. https://nginx.org/en/docs/http/ngx_http_ssl_module.html
- [8] libdatachannel Project. C/C++ WebRTC network library. <https://github.com/paullouisageneau/libdatachannel>
- [9] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara, and K. Cohn-Gordon. The Messaging Layer Security (MLS) Protocol. RFC 9420, 2023.